SELL MODULE

Technical & Functional Documentation

GROO Retail ERP (Woven Essence) · Next.js 15 + MongoDB

Screens: Delivery Challan · POS · POS Return

Prepared: 21-10-2026

Prepared by: _____ Reviewed by: _____

Contents

1. Overview & sidebar
2. Architecture at a glance
3. Screen: POS list (Sell → POS)
4. Screen: POS billing till (/admin/pos/add)
5. Flow: POS sale, step by step
6. POS print (Receipt 80mm / A5 / A4)
7. Screen: POS Return
8. Screen: Delivery Challan
9. Data models
10. API reference
11. Roles & Permissions
12. Library helpers
13. Configuration
14. Recent changes (October 2026)
15. Known issues & open points
16. How to run, test and debug

All paths are relative to the project root `wovenessenceV2/`. Line numbers are approximate (as of 21-10-2026) and point to the live code; several files keep older versions commented out above the live code.

1. Overview & sidebar

The Sell module covers retail billing at the counter (POS), customer returns/refunds (POS Return) and outward Delivery Challans to customers. Stock is held per physical unit in the `barcodeLabel` collection; selling a barcode marks it SOLD (or reduces a batch's quantity), returning puts it back IN_STOCK.

Sidebar label	Route	Main component
Delivery Challan	`/admin/transaction/sell/deliverychallan`	ListView / TransactionFormView / SellChallanView
POS	`/admin/transaction/sell/pos`	ListView + PosInvoiceView / PosPaymentsView popups
(POS till - opened from POS list "Add")	`/admin/pos/add`	PosTill
POS Return	`/admin/transaction/sell/pos-return`	ListView / PosReturnForm

Sidebar definition: `config/nav.js` (Sell group, ~lines 387-434).

> **Pages that exist but are NOT in the menu** (their nav entries are commented out): Sales Invoice (`app/admin/transaction/sell/salesinvoice/` , API `/api/sell-salesinvoice`), Sales Return (`salereturn/` , `/api/sell-salereturn`), Credit Note (`creditnote/` , `/api/sell-creditnote`), B2B Invoice (`b2binvoice/` , `/api/sell-b2binvoice`). These routes only check the login session (no screen permission), number documents via `nextDocNumber` and do not move stock.

2. Architecture at a glance

Every screen follows the same layering:

Layer	Where	Notes
Page	`app/admin/transaction/sell/**/page.jsx` , `app/admin/pos/add/page.jsx`	Thin: a config object + one component.
Component	`components/*.jsx`	Generic (ListView, TransactionFormView) or module-specific (PosTill, PosReturnForm...).
API route	`app/api/**/route.js`	Session + permission check, validation, DB writes. Stock changes go through `lib/inventory.js` inside a MongoDB transaction.
Model	`models/*.js` , `lib/barcodeLabel.js`	Mongoose models; collection names are lower-case (e.g. `posinvoice`).
Helpers	`lib/*.js`	inventory, docnumber, loyalty, listSort, screenPermission, refLabels...

Shared building blocks

- **`components/ListView.jsx`** - generic list screen: toolbar (column visibility, search, CSV/Excel/PDF export, Add), filters, summary cards, action menu, and (opt-in) **numbered pager + sortable headers** (`cfg.pager: 'numbered'` , `cfg.sortable: true` ; live code from ~line 2532, pager logic ~2655).

- **`components/NumberedPager.jsx`** - the pager footer ("Showing a-b of N", Rows 10/25/50/100, |< < 1 2 3 > >|) and `SortHeader`.
- **`lib/listSort.js`** - server side of header sorting: `sortedPage(Model, filter, sp, {page, perPage, keys, computed, refs})`. Returns `null` when no `?sortBy=` is sent, so each route keeps its default query untouched.
- **`components/TransactionFormView.jsx`** - card-driven transaction entry form (used by Delivery Challan Add/Edit).
- **`lib/screenPermission.js`** - Roles & Permissions checks (`screenDenial`, `screenDenialAny`).
- **`lib/inventory.js`** - the stock engine (scan rules, sell, return, transactions).

3. Screen: POS list (Sell → POS)

Item	Detail
Page	`app/admin/transaction/sell/pos/page.jsx`
Component	`ListView` with config: `endpoint:'/api/sell-pos'`, `pager:'numbered'`, `sortable:true`, `addHref:'/admin/pos/add'` + `addNewTab:true` (the till opens in its own browser tab).
Summary cards	Total Invoices, Total Amount, Collected (break-down Cash / UPI / Bank / Return / Net), Qty Sold (Pcs / Mtrs). Filled from the API's `summary` object, computed over the whole filtered set (not just the page).
Filters	Invoice No, Start Date, End Date (both default to **today**).
Columns	Date (bill date + time from `createdAt`), Invoice No, Counter, Customer Name, Customer Contact, Amount (with Paid / Part Paid and payment-method icons from `payments[]`), Total Items, Sales Person.
Row actions	View (popup: `PosInvoiceView`), View Payments (popup: `PosPaymentsView`), Print Invoice (print page).

Components

- **`components/PosInvoiceView.jsx`** - read-only invoice: header with invoice barcode, share buttons (WhatsApp / Instagram / Email via `lib/shareRoutes.js`), Print (new tab), items with image hover, payment table, sell/staff notes. In the popup its button reads "Close".
- **`components/PosPaymentsView.jsx`** - invoice / payment / customer cards + payments table; **Edit Payment** opens `MultiplePayDialog` and saves via `PUT /api/sell-pos/<id>`. After an edit the list re-loads (`reloadKey`).
- **`components/MultiplePayDialog.jsx`** - split payment: Cash, UPI/Card (+ adds more UPI rows), Bank Deposit; shows Payable, Paying, Cash Return, Balance; Sell note / Staff note. Shared by the till and Edit Payment.

Direct-link pages also exist: `pos/view/[id]` (PosInvoiceView) and `pos/payment/[id]` (PosPaymentsView).

API used by the list

Call	What it does
`GET /api/sell-pos` app/api/sell-pos/route.js ~51-391	Permission **POS read**. Filters business/location/finYear/date range/invoice no; summary cards default to today. Three aggregates: totals, `paidBy` buckets (Cash/UPI/Bank/Return/Net) and `qtyBy` (PC/MTR via `uomTypeOf`). Sorting: plain keys `invoiceNo, totalAmount`; computed keys `date, counterName, customerName, customerContact, salesPersonName, totalItems`. Rows get business/location/customer/counter/sales-person names.
`GET /api/sell-pos/[id]`	One invoice (POS read on the bill's business) with names, per-line sales person and the `seller` letterhead (location fields, falling back to business). Used by View, Payments and Print.
`PUT /api/sell-pos/[id]`	POS update. Edits **payments and notes only**; recomputes `paid`, `sellDue`, `paymentStatus`. Never changes the invoice number. Does not touch stock.
`DELETE /api/sell-pos/[id]`	POS delete. Removes the document only (see Known issues).

4. Screen: POS billing till

Item	Detail
Page	`app/admin/pos/add/page.jsx` (server component, tab title "POS", wrapped in Suspense)
Component	`components/PosTill.jsx` (~1530 lines, full-screen overlay)
Opened from	POS list "Add" (new tab), with `?business=&location=&finYear=` in the URL

Layout

- **Header:** ← Back to POS list (keeps scope, ~line 1325), the till's **own** Business and Location selects (independent of the top bar), clock, Calculator (safe expression evaluator `calcEvaluate`), "Main Reports" (opens Master Stock Report in a new tab).
- **Row 2:** Sale date, Sales Person, Customer (searchable; quick-add), Cashier (read-only, from `/api/auth/me`).
- **Row 3:** Billing type (Cash / Credit / Export / COD), scan box with suggestions, Cash Counter, **Return / Refund** (opens POS Return in a new tab), **Exchange** checkbox.
- **Line table (~line 1343):** Sl, Stock (Issue − / Addition + ticks), Barcode No, **Item Name**, Print Description, Item Code, HSN, GST%, Qty (stepper capped at the unit's closing stock, shows PCS/MTR), RSP Price, Disc %, Disc Amt, Net Amount, Sales Person (per line), Image, remove.
- **Footer:** Qty (Pcs/Mtrs), Bill Value, Total Discount, Sub Total, Taxable Amt (+ included tax), Net Amount; Line-wise Discount % / Amt; Shipping; loyalty strip; Hold, Held(n), Multiple Pay, Total Payable, Clear Screen, Recent Transactions.
- **Keys:** Enter / F9 / Tab in the scan box scans; a hardware scanner is caught anywhere (`useScanner`); ↑/↓ moves the highlighted line; **Esc** opens/closes Multiple Pay.

APIs called by the till

Call	Purpose	Permission
`POST /api/barcode/scan` (intent `SELL` or `POS_RETURN`)	Resolve a scanned code to a stock unit via `lib/inventory.scanBarcode`; returns `shape(unit)`.	POS read/create/update
`GET /api/inventory-barcode-list`	Scan-box suggestions.	Any of BARCODE_LIST_SCREENS
`GET /api/item`, `/api/item/<id>/detail`	Fallback when the code is not a barcode (loose item line).	ITEM_LOOKUP_SCREENS (incl. POS)
`GET /api/options?ref=business | companylocations | contact-type-customer | salesperson`	Dropdowns. Sales persons follow the **till's** business/location (19-10-2026).	-
`GET /api/pos-counter`	Cash counter dropdown (inactive counters hidden).	COUNTER_PICKER_SCREENS
`GET /api/customer`, `POST /api/customer {quick:true}`	Customer search / quick-add.	Customers read / create
`GET /api/loyalty`	Points balance, allowed redemption (read only).	POS read/create/update
`GET/POST /api/pos-hold`, `GET/DELETE /api/pos-hold/[id]`	Hold / list / resume held bills (no stock reservation).	POS read/create

`GET /api/sell-pos/recent`	Exchange: last 20 invoices containing a barcode/code.	POS read or create
`POST /api/sell-pos`	**Save the sale** (section 5).	rbac POS_SELL + POS create

5. Flow: POS sale, step by step

#	Step	What happens (code)
1	Open till	Business/location/finYear from the URL; otherwise the default business and its first location. finYear falls back to `'2026-2027'`. Changing business clears customer and sales person.
2	Scan	`addScanned` (~line 744). Normal line: `POST /api/barcode/scan` intent SELL with the already-scanned list (no duplicates). Server rule: unit must be **IN_STOCK at this location**, not in transit, not void. If `BARCODE_NOT_FOUND` the item master is tried (line without barcode). **Exchange** ticked + first line: `/api/sell-pos/recent` picker; chosen sale is scanned with intent POS_RETURN; the line is a **return line** (red, negative).
3	Line added	`addBarcodeUnit` (~line 872): qty 1, `closing` = unit qty (caps the stepper), `rsp = trunc(offerPrice || rsp)`, item name, uomType, batchType, grcNo, header sales person.
4	Edit lines	Qty ≥ 1 and ≤ closing; RSP whole rupees; Disc % 0-100; line-wise discount applies to all lines; shipping dialog.
5	Totals	`lineTotal = rsp × qty × (1 - disc%)` (negative for return lines). **RSP includes GST** - tax is backed out for display, never added. `netAmount = trunc(billValue + shipping)`.
6	Loyalty	`GET /api/loyalty` on customer/amount change; cashier may redeem up to `allowedPoints`; points earned today usable tomorrow. Payable = net - redeemed value.
7	Hold (optional)	Hold posts the till state to `/api/pos-hold` and clears the till; resume = GET then DELETE.
8	Pay	Multiple Pay / Esc → `submitPayment` (~line 1305) → `POST /api/sell-pos` with business, location, finYear and `data` {date, customer, counter, billingType, items, payments, notes, shipping, totalAmount, paid, loyaltyRedeemPoints}.
9	Server validation	Lines normalised (`normaliseLines`). Loyalty re-checked (409 `LOYALTY_NOT_AVAILABLE` if over); redeemed points become a payment row. `sellDue` and `paymentStatus` (Paid / Part Paid / Unpaid).
10	Transaction	`withTransaction`: load units (422 BARCODE_NOT_FOUND, 409 BARCODE_UNAVAILABLE); invoice number = `nextDocNumber(PosInvoice,'invoiceNo','POS',...)` (prefixed "POS" if it starts with a digit); `PosInvoice.create`; loyalty ledger rows (REDEEM / EARN); **stock**: `sellUnits` - full units become **SOLD** (billingId, billingNo, soldAt), batches reduce `qtyNum`; every sale pushed to `sales[]`; `StockMovement POS_OUT`. Stock Issue/Addition ticks also write a `StockAdjustment` register row.
11	Print	Response `{ok,id,invoiceNo}`; till redirects to `/admin/transaction/sell/pos/print/<id>` (~line 1316).

6. POS print (Receipt 80mm / A5 / A4)

Item	Detail
Page	`app/admin/transaction/sell/pos/print/[id]/page.jsx` (loads `GET /api/sell-pos/<id>`)
Paper toggle	Receipt 80mm / A5 / A4; the choice is remembered per PC in `localStorage['posPrintFormat']`.
Logos	`bill-template/posinvoice-temp.png` (LOGO) and `bill-template/posinvoice-temp1.png` (LOGO2, added 19-10-2026). Imported, so Next bundles them.
Constants	TAGLINE, THANK_YOU, DEFAULT_TERMS (overridden by the location's terms), FOOTER_NOTE "TUESDAY HOLIDAY".

Maths	`lineNet` (stored netAmount → lineTotal → recompute); `lineTax` backed out of the inclusive price; CGST = SGST = tax/2; round off = total - shipping - lines; Pcs/Mtr by uomType; tax summary grouped by HSN + rate.

> **The 80mm (4-inch) receipt layout is FROZEN by the client since 12-10-2026 (B2C and B2B).** Do not change its columns, order or spacing. Two lines per item: (Sl, Description, GST, HSN) then (Qty, Bar Code, MRP, Disc%, Net Amt).

A5 / A4 (`SheetInvoice`): same information as the receipt, laid out as a page with no cell boxes (dashed rules), both logos side by side, and the client's column order: **Sl, Item Description, Barcode, Qty, HSN, GST, MRP, Disc%, Disc Amt, Net Amt** (`lineDisc` = MRP × Qty - line net).

Printing mechanics: `setPageSize()` writes an `@page` rule (receipt: 80mm × measured height; sheets: A4/A5 portrait) on `beforeprint`. The document is rendered twice - on screen (`.pos-screen`) and as a print-only copy portalled to `<body>` (`.print-doc.pos-print-copy`) because `globals.css` hides everything else when printing. Close button returns to the POS list.

7. Screen: POS Return

Item	Detail
List	`app/admin/transaction/sell/pos-return/page.jsx` - ListView (`/api/sell-pos-return`), numbered pager + sorting. Columns: Date, Invoice No, Parent Invoice, Customer, Payment Status, Items, Total, Refunded, Reason, Processed By, Location.
Add	`.../pos-return/add/page.jsx` → `components/PosReturnForm.jsx` (also opened from the till's Return / Refund button with the customer pre-filled).

PosReturnForm behaviour

- Business/location/finYear/customer from the URL, else the top bar.
- With a customer: lists their bills; picking one loads it with all returnable lines ticked.
- Find a sale by invoice number or by scanning an item; scanning then ticks lines (local match first, then `/api/barcode/scan` intent LOOKUP).
- Non-returnable lines are greyed with the reason. Reason: Damaged / Wrong Size / Wrong Item / Not Satisfied / Other. Refund mode: Cash. Refund amount optional (blank = full).
- Close → POS Return list. After save: "Return completed" panel.

API

Call	What it does
`GET /api/sell-pos-return`	List; POS Return read; sorting keys date, invoiceNo, parentInvoice, paymentStatus, returnedCount, totalAmount, refundAmount, reason, processedBy; refs customer, location.
`GET /api/sell-pos-return/lookup`	`?customer=` last 50 bills with returnable counts; `?invoice=` / `?barcode=` the invoice with returnable lines, blocked reasons, prior returns. rbac POS_RETURN + screen read/create.
`POST /api/sell-pos-return`	The return (flow below).
`GET/PUT/DELETE /api/sell-pos-return/[id]`	Read / replace items / delete (delete does not reverse stock).

Return flow

#	Step	Detail
1	Checks	rbac POS_RETURN + screen create; parent invoice valid and of this business; at least one barcode; reason required.
2	On the invoice	Every scanned code must be a line of that invoice, else 422 NOT_ON_INVOICE.
3	Not returned before	No earlier PosReturn on the same invoice for that barcode, else 409 ALREADY_RETURNED.
4	Refund cap	`refundable = Σ net × (1+gst%)` ; a higher refund is refused.
5	Transaction	Re-check clash; number = `nextDocNumber(PosReturn,'invoiceNo','POS Return')` ; `PosReturn.create` ; `returnSoldUnits` : units back to **IN_STOCK** at the till's location, qty restored, billing fields cleared, `sales[].returnedQty` increased, `StockMovement POS_RETURN_IN` .

6	Response	201 {id, invoiceNo, refundAmount, returnedCount}. The original POS invoice is not modified; loyalty is not reversed.

8. Screen: Delivery Challan

Route	File	Renders
List	`app/admin/transaction/sell/deliverychallan/page.jsx`	ListView (numbered pager, sortable; actions Edit, Print Challan; filters Start/End date on dcDate)
Add / Edit	`.../deliverychallan/add/page.jsx`, `.../[id]/page.jsx`	TransactionFormView with `FORM`
Print	`.../deliverychallan/[id]/challan/page.jsx`	SellChallanView → ChallanDocument
Form spec	`.../deliverychallan/form.js`	Fields, scan card, item grid, totals card (also imported by the API)

- **Header fields:** customer (required), customer GSTN (required, read-only), address, DC date (default today), stock point, logistic, sales term (After/Before Tax), sales group, sales person (ref **agent**), sales ledger, agent, customer waybill (file).
- **Item scan** in TransactionFormView resolves codes against the **Item master** (`GET /api/item`), not the barcode engine.
- **`components/ChallanDocument.jsx`** - shared printed challan (also used by Stock Transfer): Detailed / Non-detailed (driver copy hides prices) toggle, Qty/PC/MTR summary via `lib/challan.js`, Print, optional Close.

Call	What it does
`GET /api/sell-deliverychallan`	SELL_DELIVERY_CHALLAN read; filters business/location/finYear/customer/date/unconverted/search; sorting keys deliveryChallanNo, customerMobile, customerGstn, totalQty, createdAt; refs customer, logistic.
`POST /api/sell-deliverychallan`	Create: `validate(FIELDS)` (FIELDS derived from form.js), items as sent, number = `nextDocNumber(DeliveryChallan,'deliveryChallanNo','Delivery Challan')`. **No stock movement.**
`GET/PUT/DELETE /api/sell-deliverychallan/[id]`	Read (with labels for the print) / update (number never overwritten) / delete - each checked against the challan's own business.

9. Data models

Model file	Collection	Key fields
`models/PosInvoice.js`	posinvoice	businessId, locationId, finYear, date, invoiceNo, counterId, customerId, customerContact, customerSnapshot, billingType, paymentStatus, shipping, totalAmount, paid, sellDue, payments[], loyaltyPointsRedeemed/Earned, loyaltyAmount, sellNote, staffNote, items[] (barcodeNo, itemCode, itemName, description, hsn, gst, uom, uomType, qty, rsp, discountPct, netAmount, salesPerson, isReturn, stockIssue, stockAddition)
`models/PosReturn.js`	posreturn	invoiceNo, parentInvoice, parentInvoiceId, customer, totalAmount, refundAmount, refundMode, paymentStatus (Refunded / Part Refunded), reason, returnedQty, returnedCount, processedBy, items[]
`models/DeliveryChallan.js`	deliverychallan	customerId, customerGstn, customerAddress, dcDate, stockPointId, logisticId, salesTerm, salesPersonId (agent), deliveryChallanNo, totals, items[], salesInvoiceId (also written by Stock Transfer DC)
`lib/barcodeLabel.js`	barcodeLabel	One row per stock unit: barcodeNo, barcodeGenerated, oldBarcode, businessId, qtyNum, status, currentLocationId, billingId, billingNo, soldAt, **sales[]** {invoiceId, invoiceNo, qty, at, returnedQty, icBilledId, icBilledNo}
`models/StockMovement.js`	-	Movement ledger (POS_OUT, POS_RETURN_IN)
`models/StockAdjustment.js`	-	Register rows for till Stock Issue / Addition ticks
`models/PosHold.js`	poshold	Held bills (holdNo = HH:MM label), till header, items, totals
`models/LoyaltyLedger.js` / `LoyaltyPoint.js`	loyaltyledger / loyaltypoint	EARN / REDEEM rows (signed points, expiry) / loyalty settings
`models/PosCounter.js`, `SalesPerson.js`, `CashRegister.js`, Contact (customer)	-	Read-only from Sell

barcodeLabel statuses (`BARCODE_STATUS`): IN_STOCK, IN_TRANSIT, SOLD, RETURN_IN_TRANSIT, VOID, HISTORY.

10. API reference

Route	Methods	Permission (screen / action)	Purpose
`/api/sell-pos`	GET, POST	POS read / create (+ rbac POS_SELL)	List + summary; save a sale
`/api/sell-pos/[id]`	GET, PUT, DELETE	POS read / update / delete	One bill; edit payments; delete
`/api/sell-pos/recent`	GET	POS read or create	Exchange lookup
`/api/pos-hold`, `/[id]`	GET, POST, DELETE	POS read / create	Held bills
`/api/barcode/scan`	POST	POS read/create/update (SELL, POS_RETURN)	Scan a unit
`/api/loyalty`	GET	POS read/create/update	Loyalty balance

`/api/sell-pos-return`	GET, POST	POS Return read / create (+ rbac POS_RETURN)	List; process a return
`/api/sell-pos-return/lookup`	GET	POS Return read/create	Find the sale
`/api/sell-pos-return/[id]`	GET, PUT, DELETE	POS Return read / update / delete	One return
`/api/sell-deliverychallan`, `/[id]`	GET, POST, PUT, DELETE	Sell Delivery Challan read/create/update/delete	Delivery challans
`/api/inventory-barcode-list`	GET	BARCODE_LIST_SCREENS read	Scan suggestions
`/api/pos-counter`	GET	COUNTER_PICKER_SCREENS read	Counters
`/api/customer`	GET, POST	Customers read / create	Search / quick-add
`/api/options`	GET	-	Dropdown lists

11. Roles & Permissions

Permissions are saved per role and per business in `models/RolePermission.js`, keyed by the sidebar href, and edited under Staff Management → Roles & Permissions. Server checks live in `lib/screenPermission.js`:

- **Super Admin** is never restricted.
- A role with **no saved matrix anywhere** is allowed (legacy behaviour).
- Once a role has a saved matrix, any screen not ticked is denied (403 with `{error, code}`).
- `lib/rbac.js` `requirePermission(PERMISSIONS.POS_SELL / POS_RETURN)` is an older, separate check still applied to sale and return.
- The browser hides buttons with `useScope().can(href, action)` (`components/ScopeContext.jsx`); the API check is the real enforcement.

Constant	href	Guards
SCREENS.POS	/admin/transaction/sell/pos	/api/sell-pos*, /api/pos-hold*, /api/barcode/scan (SELL/POS_RETURN), /api/loyalty; also part of item lookup, barcode list and counter picker screen lists. The till counts as POS create.
SCREENS.POS_RETURN	/admin/transaction/sell/pos-return	/api/sell-pos-return* (deliberately not also gated on POS)
SCREENS.SELL_DELIVERY_CHALLAN	/admin/transaction/sell/deliverychallan	/api/sell-deliverychallan*
SCREENS.CUSTOMER	/admin/contact/customer	Till customer search (read) / quick-add (create)

12. Library helpers

File	Role in Sell
`lib/inventory.js`	Stock engine: `scanBarcode` (~157) + `assertUsable` rules for SELL / POS_RETURN (~353), `withTransaction` (~80), `loadUnits`, `unitsByCode`, `sellUnits` (~795), `returnSoldUnits` (~876), `shape`, `imageUrl`, `InventoryError`.
`lib/barcodeLabel.js`	BarcodeLabel model + BARCODE_STATUS.
`lib/barcodeValue.js`	Barcode spelling helpers (`sameBarcode`, `barcodeKey` ...) so a unit answers to its number, composed value or old barcode.
`lib/barcodeUnits.js`	`uomTypeOf(uom)` → 'PC' / 'MTR'.
`lib/docnumber.js`	`nextDocNumber` - next number from the Doc Setup master (prefix, padding) with an atomic counter. Types: POS, POS Return, Delivery Challan, Stock Adjustment.
`lib/loyalty.js`	Rules, balance (FIFO + expiry), earn/redeem maths, ledger rows.
`lib/listSort.js`	Header sorting for list routes (`sortedPage`).
`lib/screenPermission.js`, `lib/rbac.js`	Permission checks.
`lib/refLabels.js`	Turns id fields into display names for list rows / print.
`lib/validate.js`	`validate(fields, body)` - coerces, checks required, **drops unknown keys**.
`lib/challan.js`	Challan line normalising and Qty/PC/MTR summary.

`lib/apiError.js`	`handler()` maps InventoryError / AuthzError to HTTP codes.
`components/useScanner.js`, `useOptions.js`	Scanner hook + barcode lookup; dropdown loader (its `params` override the top-bar business/location).

13. Configuration

- **Doc Setup** (Masters → Doc Setup) must have document types **POS**, **POS Return**, **Delivery Challan**, **Stock Adjustment** with prefixes for the business/location/finYear; otherwise numbers come out as bare padded digits.
- **Loyalty**: Masters → Loyalty Point (active flag, points-to-rupee, earning rule, min/max redemption).
- **Counters**: Masters → Cash Counter Master. **Sales persons**: Staff Management → Sales Persons (per business; the till lists the sales persons of the business selected *on the till*).
- **Print**: logos in `bill-template/`; store address/terms from Company Location (falls back to Business).
- **Database**: `.env` → `MONGODB_URI`. The app caches the connection, so **restart the dev server** after changing it.
- **Product images**: `barcodeLabel.imageUrl` = `/august_8A_images/<barcode>.jpg`, served from `public/august_8A_images/` on the server.

14. Recent changes (October 2026)

Date	Change
05-10	POS list cards and filters open on today's date; part-sale of batch barcodes.
12-10	Collected card shows Bank; till goes straight to print after save; 80mm receipt frozen by the client.
13-10	Collected card Return / Net; three paper sizes (Receipt / A5 / A4).
17-10	Roles & Permissions on loyalty and barcode scan (SELL / POS_RETURN).
19-10	Numbered pager + header sorting on all three Sell lists; Item Name column on the till; sales persons follow the till's own business; A5/A4 column order, no boxes, second logo.
21-10	Close buttons: Delivery Challan Add/Edit and print, POS Return form.

15. Known issues & open points

Found while reading the code for this document. **Documented only - not fixed.** Confirm with the business before changing any of them.

#	Issue	Detail / where
1	Delivery Challan Add/Edit probably cannot be saved	`customerGstn` is required and read-only in `deliverychallan/form.js`, but nothing fills it (no `fillFrom`), so `validate()` should reject with 422. Also `customerMobile` / `totalQty` are not in FIELDS, so this screen never stores them.
2	POS edit/delete does not touch stock	`DELETE /api/sell-pos/[id]` removes the bill but leaves its barcodes SOLD, writes no reversing movement and keeps loyalty rows. Same for deleting a POS Return (stock stays IN_STOCK).
3	Refund cap may be too high	Return routes compute `net × (1 + gst%)`, but prices are GST-inclusive everywhere else.
4	Returns do not reverse loyalty	`LOYALTY_KIND.REVERSAL` exists but is never written.
5	Cash register expected balance	Closing a register sums all POS `totalAmount` (not only cash) and subtracts return `totalAmount` rather than `refundAmount`.
6	Number gaps	`nextDocNumber` for POS / POS Return runs outside the transaction session, so a rolled-back sale still uses a number.
7	Hard-coded financial year	The till falls back to `'2026-2027'` when finYear is not in the URL.
8	Unused counter settings	`PosCounter.invoiceLayout` / `repeatInvoice` are not read by the print page.

16. How to run, test and debug

1. `npm install`, set `.env` (MONGODB_URI, AUTH_SECRET), `npm run dev` → `http://localhost:3000`.
2. Log in, choose Company / Location / Financial Year in the top bar.
3. Sell → POS → Add opens the till in a new tab. Scan a barcode that is IN_STOCK at that location, pay, check the print.
4. Check the result in MongoDB: `posinvoice` (new bill), `barcodeLabel` (status SOLD / qtyNum reduced, `sales[]` entry), `loyaltyledger`.
5. Sell → POS Return → Add, enter the invoice number, tick a line, save; the barcode returns to IN_STOCK.

Where to look first

- "Barcode not found / not in stock" at the till → `lib/inventory.js` `scanBarcode` / `assertUsable`, and the unit's status/location in `barcodeLabel`.
- Wrong invoice number → Doc Setup entry for "POS" and `lib/docnumber.js`.
- 403 "no permission" → the role's ticks in Roles & Permissions (`lib/screenPermission.js`).
- Print looks wrong → `app/admin/transaction/sell/pos/print/[id]/page.jsx` (remember the 80mm layout is frozen).
- List sorting/paging → `components/ListView.jsx` + `lib/listSort.js`.
- Every file changed recently is listed by date in `change.md` at the project root.